Web Server Fingerprinting Tool

1. Objective

The main aim of this project is to understand how communication happens between a client and a web server over a network. Along with that, we built a tool that can identify what type of server software is being used by different websites. This helps us connect theoretical networking concepts with a practical implementation.

2. Problem Statement

Whenever we access a website, we usually don’t know what kind of server is handling our request in the background. However, knowing this information can be useful for analysis, debugging, and even basic security understanding.

In this project, the goal is to design a system that connects to a website and identifies the server software by analyzing the response it sends back.

3. Approach

To solve this problem, we followed a simple but effective approach.

First, the program takes a list of websites as input. Then, for each website, it creates a connection using sockets. After establishing the connection, it sends a request to the server and waits for a response.

Once the response is received, the program looks for specific information inside it, especially the part that reveals details about the server. If that information is not directly available, the program tries to make an educated guess based on known patterns.

This entire process is done for multiple websites at the same time using threading, which improves efficiency.

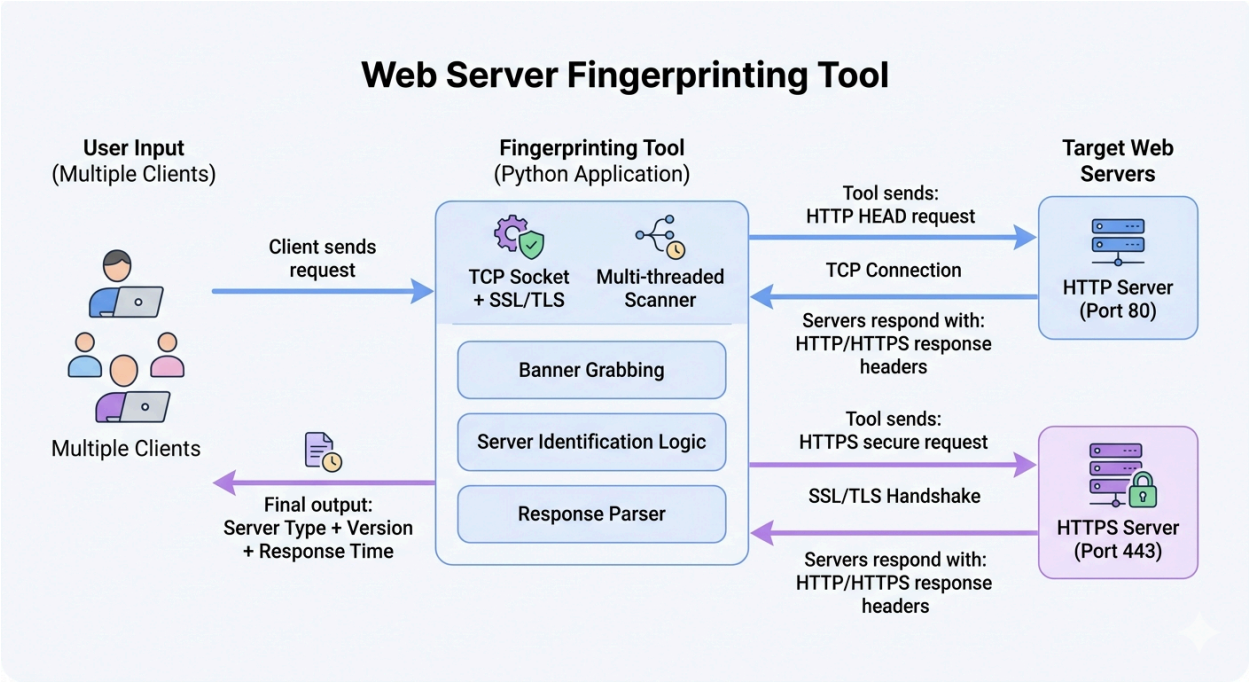
4. System Architecture

The project follows a client-server model.

The client (our program) initiates a connection and sends a request. The server processes that request and sends back a response. The client then analyzes this response to extract useful information.

Flow of execution:

Client → Connection → Server → Response → Data Analysis → Result



5. Technologies Used

The implementation was done using Python.

Socket programming was used to establish direct communication with servers.
SSL/TLS was used to handle secure HTTPS connections.
Threading was used so that multiple websites could be scanned at the same time without delay.

6. Implementation Details

The core of the project is based on creating TCP socket connections. Two types of connections are handled:

- HTTP connections using port 80
- HTTPS connections using port 443 with SSL

Instead of requesting the full webpage, a HEAD request is used. This allows us to receive only the header information, which is enough for our purpose and also reduces unnecessary data transfer.

The response received from the server is processed line by line to find relevant details. If the server hides its identity, additional checks are performed to still identify it based on known patterns.

The program also includes proper error handling to deal with situations like connection failures, timeouts, or invalid inputs.

7. Setup and Execution

To run the project:

1. Make sure Python is installed on the system
2. Save the program as main.py
3. Open terminal or command prompt
4. Navigate to the project folder
5. Run the command:
   python main.py
6. Enter the list of websites when prompted, separated by commas

Example:
google.com, github.com, example.com

8. Sample Output

The output displays the detected server information for each website.

For example:

google.com
HTTP Server: gws
HTTPS Server: gws

github.com
HTTP Server: nginx
HTTPS Server: nginx

The program also shows the time taken for each scan.

9. Performance Analysis

The program was tested with multiple websites.

It was observed that HTTP responses are generally faster, while HTTPS takes slightly more time due to the additional SSL handshake process. Using threading helped reduce the overall execution time significantly when scanning multiple websites.

Most websites responded within a short time range, making the tool efficient for practical use.

10. Error Handling and Improvements

The program was designed to handle common issues that may occur during network communication.

It manages timeouts so that the program does not get stuck waiting for a response.
It handles cases where the server does not respond or refuses connection.
It also deals with situations where the server hides its identity.

Small improvements like retrying failed requests and limiting waiting time were added to make the tool more stable.

11. Conclusion

This project helped in gaining a clear understanding of how client-server communication works at a low level. By implementing socket programming directly, we were able to see how requests and responses are handled in real time.

The tool successfully identifies server software for different websites and demonstrates how fingerprinting can be done in a simple yet effective way.

12. Future Scope

There are several ways this project can be extended in the future.

More advanced techniques can be added to improve detection accuracy.
Results can be stored in files for further analysis.
A graphical interface can be developed to make the tool easier to use.

13. References

Python official documentation
Basic networking and HTTP protocol resources

14. Screenshots of Output

```
C:\Tanu\3 YEAR\CN>python fingerprinting.py
Enter websites (comma separated): www.google.com , www.github.com , www.example.com , www.netflix.com

--- www.github.com ---
[HTTP]  Server header not found
[HTTP]  Identified: Unknown
 HTTP Time:  0.1 sec
[HTTPS] Server header not found
[HTTPS] Identified: Unknown
 HTTPS Time: 0.16 sec
 HTTPS is 0.06 sec slower than HTTP
 Preview: HTTP/1.1 301 Moved Permanently Content-Length: 0 Location: https://www.github.com/ connection: close

--- www.example.com ---
[HTTP]  Server: cloudflare
[HTTP]  Identified: Cloudflare
 HTTP Time:  0.08 sec
[HTTPS] Server: cloudflare
[HTTPS] Identified: Cloudflare
 HTTPS Time: 0.2 sec
 HTTPS is 0.12 sec slower than HTTP
 Preview: HTTP/1.1 200 OK Date: Tue, 17 Mar 2026 09:17:03 GMT Content-Type: text/html Connection: close CF-RAY: 9ddae6ffab277e53-BOM Last-Modified: Mon, 16 Mar 2026 20:51:17 GMT A
llow: GET, HEAD Accept-R

--- www.google.com ---
[HTTP]  Server: gws
[HTTP]  Identified: Google Web Server
 HTTP Time:  0.38 sec
[HTTPS] Server: gws
[HTTPS] Identified: Google Web Server
 HTTPS Time: 0.45 sec
 HTTPS is 0.07 sec slower than HTTP
 Preview: HTTP/1.1 200 OK Content-Type: text/html; charset=UTF-8 Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-_mSJO53OgCijNmQIUbhriA' '
strict-dynamic' 'report-sampl

--- www.netflix.com ---
[HTTP]  Server header not found
[HTTP]  Identified: Unknown
 HTTP Time:  0.6 sec
[HTTPS] Server header not found
[HTTPS] Identified: Unknown
 HTTPS Time: 0.65 sec
 HTTPS is 0.05 sec slower than HTTP
 Preview: HTTP/1.1 301 Moved Permanently Location: https://www.netflix.com/ Content-Length: 0 Via: 1.1 i-00f3d91eac498343c (us-west-2) X-Xss-Protection: 1; mode=block; report=http
s://www.netflix.com/ichnaea
```

```
--- www.netflix.com ---
[HTTP]  Server header not found
[HTTP]  Identified: Unknown
 HTTP Time:  0.6 sec
[HTTPS] Server header not found
[HTTPS] Identified: Unknown
 HTTPS Time: 0.65 sec
 HTTPS is 0.05 sec slower than HTTP
 Preview: HTTP/1.1 301 Moved Permanently Location: https://www.netflix.com/ Content-Length: 0 Via: 1.1 i-00f3d91eac498343c (us-west-2) X-Xss-Protection: 1; mode=block; report=http
s://www.netflix.com/ichnaea

 Total Scan Time: 1.25 seconds

 Observations:
- HTTPS is slower due to SSL/TLS handshake
- Multi-threading reduces total scan time
- Some servers hide their identity for security

C:\Tanu\3 YEAR\CN>
```